

AI+ROS 机器人开发入门与实战： Navigation · MoveIt · YOLO

第 1 篇 Ubuntu 基础（郭超群）

第 1 章 Ubuntu 开发环境概述

参考链接 <https://qielongfei.work/Markdown/Ubuntu/chapter1-3.html>

1 教学资源 / 2 Ubuntu / 1.3 Ubuntu 基础知识

1.1 Ubuntu 简介

欢迎来到 Ubuntu 的世界！无论你是计算机专业的学生，还是对开发环境感兴趣的初学者，Ubuntu 都将是探索计算机科学和软件开发之旅中一位强大而友好的伙伴。它就像一座连接初学者与专业开发者之间的桥梁，既保持了开源系统的强大能力，又提供了令人惊喜的易用性。

Ubuntu（乌班图）是一个带有可视化操作界面的 GNU/Linux 操作系统。

① 什么是 linux？

Linux 内核，其发音为 ‘lee-nucks’，是 Ubuntu 操作系统的核心。Linux 是一个操作系统，如同 Windows(7, 10, 11) 和 Mac OS。操作系统主要作用是桥接软件和硬件其在整个计算机系统角色见下图 1-1。

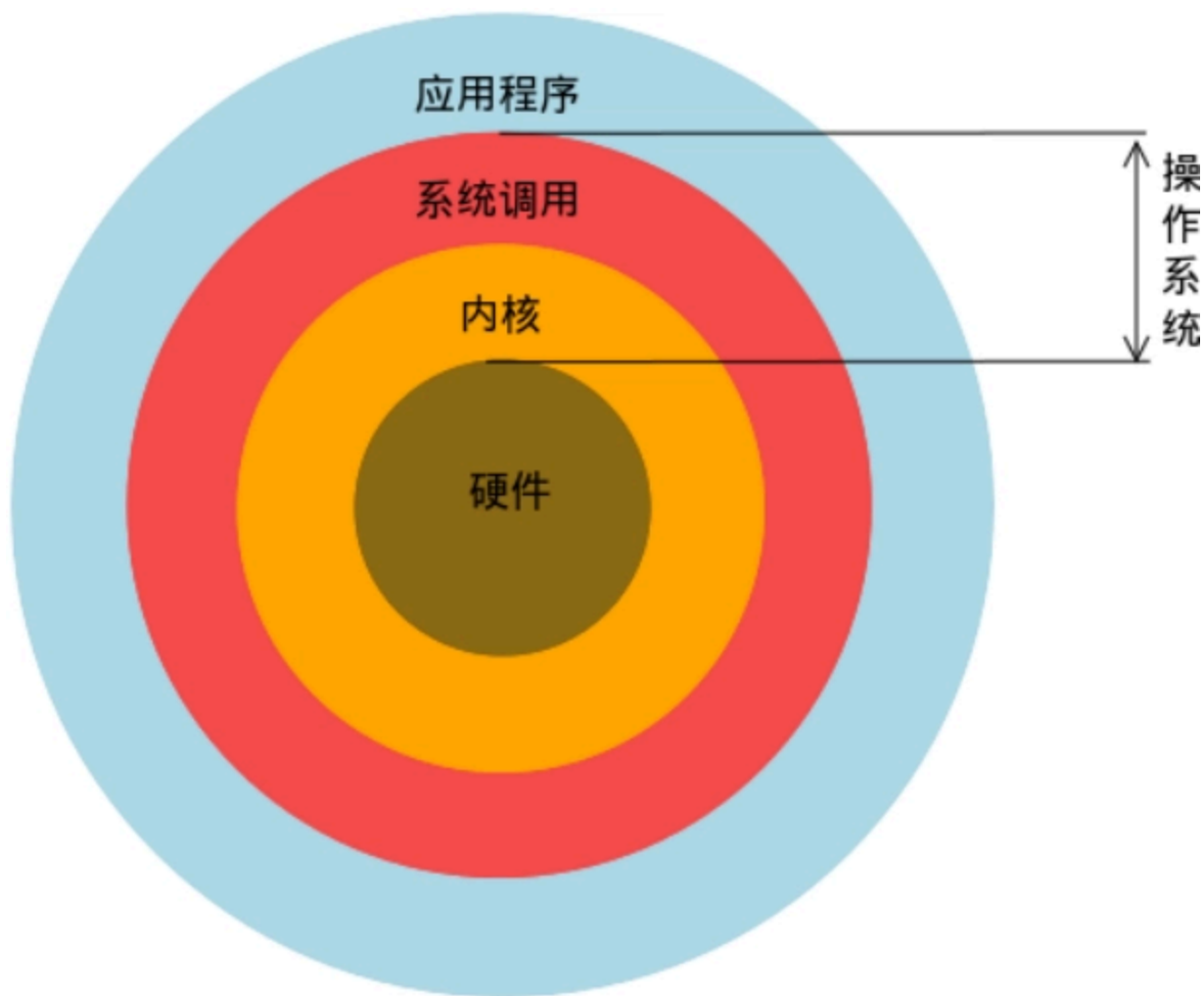


图 1-1 linux 操作系统

Linux 主要是系统调用和内核这两层。直观地看，常用的操作系统还包含在其上运行的应用程序，比如文本编辑器、浏览器、电子邮件等。

②什么是 GNU？

GNU 是一个操作系统的名称和项目，操作系统基础的“身体”部分，其核心由一系列基础且关键的软件工具构成。提供了让内核发挥作用所必需的核心工具集，例如命令行环境 (Bash)、编译器 (GCC)、基础库等。

简单说，GNU 提供了构建一个完整操作系统所必需的基础软件环境，而 Linux 是运行在这个环境里的核心引擎。两者结合，才形成了像 Ubuntu 这样的完整系统。

1.1.1 发展历史

每个成功的项目都有一个起点，Ubuntu 的故事始于 2004 年。当时，南非企业家马克·沙特尔沃思创立了 Canonical 公司，并发布了第一个 Ubuntu 版本：Ubuntu 4.10 “Warty

Warthog”。这个有趣的名字开启了 Ubuntu 每半年发布一个新版本的传统，每个版本都有一个独特的动物代号。

关键发展阶段：

早期探索期（2004-2010）：Ubuntu 基于成熟的 Debian 系统，但更注重易用性和用户体验。它迅速赢得了“最适合 Linux 初学者”的声誉。

创新实验期（2011-2017）：Ubuntu 推出了自己独特的 Unity 桌面环境，试图为桌面、手机和平板提供统一的操作体验。这一时期的 Ubuntu 展示了大胆的创新精神。

回归与成熟期（2018 至今）：从 Ubuntu 18.04 开始，系统重新采用 GNOME 桌面，同时发展出完善的长期支持（LTS）体系。如今，Ubuntu 不仅在个人电脑上广泛使用，更成为云计算、服务器和物联网领域的重要力量。

冷知识：Ubuntu 每两年会发布一个长期支持版本（LTS），这些版本提供长达 5 年的更新支持，特别适合需要稳定性的学习环境和企业应用。初学者通常建议从最新的 LTS 版本开始学习。

1.1.2 桌面环境与版本选择

1.1.3 Ubuntu 安装方式（虚拟机、双系统、纯 Ubuntu）

1.2 Ubuntu 文件和目录

1.2.1 Linux 文件类型

1.2.2 目录结构与 FHS 标准

第 2 章 终端与 Shell

2.1 终端类型与使用方式

2.1.1 仿真终端与文本终端

2.1.2 bashrc 与环境变量

2.2 ROS 开发常用命令

2.2.1 source

2.2.2 cd/mkdir/ls

2.2.3 grep/find

2.3 Shell 编程基础

2.3.1 Shell 变量与参数

2.3.2 流程控制

2.3.3 Shell 脚本

2.4 ROS 用 Shell 脚本示例

2.4.1 一键启动多节点

2.4.2 自动 source 工作空间

2.4.3 rosbag 录制脚本

2.4.4 批量 kill 节点

第3章 Ubuntu 系统管理与软件包管理

3.1 归档与压缩

3.1.1 归档

3.1.2 压缩

3.2 权限管理与 sudo 机制

3.2.1 权限定义与变更

3.2.2 sudo 机制

3.3 Ubuntu 软件包管理

3.3.1 包管理器基础

3.3.2 APT 工具详解

3.3.3 dpkg 工具

3.3.4 软件仓库与 PPA

3.3.5 ROS 依赖管理工具 rosdep

第 2 篇 ROS 基础（马睿棋）

参考链接 <https://qielongfei.work/Markdown/ROS1/chapter1-1.html>

1 教学资源 /3ROS/1 基础入门

第 4 章 ROS 概述与开发环境搭建

4.1 ROS 简介与安装

4.1.1 ROS 发展历史

4.1.2 ROS 设计目标和特点

4.1.3 ROS 安装

4.2 ROS 集成开发环境

4.2.1 终端工具 Terminator

4.2.2 VS Code

第 5 章 ROS 架构与核心机制

5.1 ROS 文件系统与功能包

5.1.1 ROS 文件系统

5.1.2 ROS 功能包

5.1.3 工作空间结构

5.2 ROS 计算图

5.2.1 主机和节点

5.2.2 消息和服务

5.2.3 话题和参数服务器

5.2.4 消息记录包

第 6 章 ROS 通信机制

6.1 ROS 通讯机制

6.1.1 话题通信

6.1.2 服务通信

6.1.3 动作通信

6.1.4 参数服务器

6.2 通讯方式特点及使用场景

第 7 章 ROS 常用工具与调试（毕新杰）

7.1 常用工具

7.1.1 RViz 和 Gazebo

7.1.2 rqt

7.1.3 rosbag

7.1.4 TF 坐标变换

7.2 ROS 系统调试

7.2.1 rostopic / rosnode / rosservice

7.2.2 rqt_graph

7.2.3 日志系统

7.2.4 常见通信与 TF 错误分析

第 3 篇 ROS 实践

第 8 章 实践 1：移动机器人自主导航（毕新杰）

8.1 机器人传感器与底层通信

8.1.1 激光雷达

8.1.2 深度相机

8.1.3 编码器

8.1.4 陀螺仪

8.1.5 串口 / CAN 通信

8.2 机器人建模与仿真

8.2.1 URDF/Xacro 建模

8.2.2 Gazebo 仿真

8.2.3 Gazebo 与 RViz 仿真联动

8.3 SLAM 建图

8.3.1 地图

8.3.2 Gmapping

8.3.3 Karto

8.3.4 Hector

8.4 自主导航与避障

8.4.1 Navigation Stack

8.4.2 move_base

8.4.3 costmap

8.4.4 Map_server&AMCL

8.5 ROS 工程规范与实践经验

8.5.1 ROS 工程目录组织

8.5.2 launch 文件组织方式

8.5.3 参数 YAML 管理

8.5.4 SLAM 地图优化

8.5.5 Navigation 调参方法

第 9 章 实践 2：机械臂与 MoveIt（柴春磊）

9.1 MoveIt 架构与配置

9.1.1 MoveIt 系统架构

9.1.2 MoveIt 安装与配置

9.2 机械臂运动规划

9.2.1 运动规划与控制接口 `ros_control`

9.2.2 MoveIt 的运动规划过程

9.2.3 MoveIt 常用 API

9.2.4 编写运动规划代码

9.2.5 抓取放置任务

9.3 MoveIt 碰撞检测

9.3.1 碰撞检测库

9.3.2 碰撞检测流程

9.3.3 连续碰撞检测

9.3.4 碰撞检测结果可视化

第 10 章 实践 3：目标检测 YOLO（柴春磊）

10.1 YOLO 架构与原理

10.1.1 目标检测任务概述

10.1.2 YOLO 系列算法发展

10.1.3 YOLO 网络结构与输出形式

10.1.4 YOLO 损失函数与训练目标

10.1.5 YOLO 推理流程与性能指标

10.2 训练自己的 YOLO 模型

10.2.1 数据集采集与标注

10.2.2 数据集格式与配置文件

10.2.3 训练环境搭建

10.2.4 模型训练流程与参数设置

10.2.5 模型评估与结果分析

10.3 YOLO 推理与模型部署

10.3.1 模型权重加载与推理流程

10.3.2 图像与视频目标检测

10.3.3 推理速度与精度权衡

10.3.4 常见部署问题与解决方法

10.4 YOLO 与 ROS 的集成应用

10.4.1 ROS 图像消息与相机节点

10.4.2 YOLO 检测节点设计

10.4.3 检测结果的 ROS 消息发布

10.4.4 RViz 中的检测结果可视化

附录 A ROS 编程基础

1. Python 语法基础

2. C++ 语法基础

附录 B ROS 构建系统与配置文件

1. XML

1.1. 常用标签与语法规则

1.2. 参数、命名空间与重用

1.3. 常见错误与排查

2. CMake

2.1. 最小 CMake 模板

2.2. 依赖声明与链接规则

2.3. 编译报错速查表

